Christie’s East Hampton · Full Platform Inventory

What Is Running, What Is Dead Weight, What to Cut

April 1, 2026 · Prepared by Manus

The Short Answer

There is no Telegram bot in this codebase. There are no old Netlify functions running. There are no Manus-scheduled background tasks firing outside the server. The platform is cleaner than it may feel from the outside. What follows is the complete honest picture of every process, every package, every integration, and every legacy thread — so the council can see exactly what is alive and what is taking up space.

Part 1 — What Is Actually Running Right Now

These are the live processes that fire when the server starts.

Process	Type	Schedule	Status
Express server	HTTP server	Always on	Live
tRPC API (`/api/trpc`)	Request handler	On demand	Live
OAuth callback (`/api/oauth/callback`)	Request handler	On demand	Live — Manus auth
Market data proxy (`/api/market-data`)	Request handler	On demand	Live — Yahoo Finance + CoinGecko
TTS — Founding Letter (`/api/tts/founding-letter`)	Request handler	On demand	Live — ElevenLabs
TTS — Market Report (`/api/tts/market-report`)	Request handler	On demand	Live — ElevenLabs
PDF generation (`/api/pdf/report`)	Request handler	On demand	**Orphaned — see below**
WhatsApp Morning Brief (`/api/whatsapp/morning-brief`)	Cron + endpoint	8:00 AM Eastern daily	**Failing — Twilio auth error**
WhatsApp Evening Summary (`/api/whatsapp/evening-summary`)	Cron + endpoint	8:00 PM Eastern daily	**Failing — Twilio auth error**
WhatsApp Test (`/api/whatsapp/test`)	POST endpoint	On demand	Wired, untested
node-cron scheduler	Background process	Starts with server	Running but both jobs failing

The Twilio problem in plain language: The cron jobs fired at 8PM last night and got a `20003` authentication error from Twilio. This means the Twilio credentials (Account SID and Auth Token) are not validating. The most likely cause is that the secrets were never successfully injected into the production environment — the secrets card UI was having issues when you provided the credentials. The code is correct. The credentials just need to be re-entered through the secrets manager and confirmed live before the next cron window.

Part 2 — Server Routes That Are Live and Serving a Purpose

Every route below is actively called by the frontend.

Route	Called From	Purpose	Keep?
`GET /api/market-data`	DashboardLayout ticker + MARKET tab rate panel	Live S&P, Gold, Silver, VIX, Treasury, BTC, Mortgage	**Yes**
`GET /api/tts/founding-letter`	ReportPage TTS Button 1	ElevenLabs audio for founding letter	**Yes**
`GET /api/tts/market-report`	ReportPage TTS Button 2	ElevenLabs audio for full market report	**Yes**
`/api/trpc/pipe.*`	PipeTab	Deal CRUD — list, upsert, delete	**Yes**
`/api/trpc/auth.*`	DashboardLayout	Login/logout state	**Yes**
`/api/trpc/system.*`	Template default	Owner notifications	**Keep — low overhead**

Part 3 — The Orphaned Puppeteer PDF Route

`GET /api/pdf/report` — This route exists in `server/pdf.ts`. It launches a headless Chromium browser (via `puppeteer-core` + `@sparticuz/chromium`), navigates to the live `/report` page, and returns a pixel-accurate PDF. It was built to solve a specific problem: html2canvas cannot render Tailwind 4's `oklch()` CSS colors correctly.

The problem: The Download button on the `/report` page no longer calls this route. It was switched to `generateMarketReport()` from `pdf-exports.ts`, which uses jsPDF (client-side). The Puppeteer route is still registered on the server and still starts a Chromium process on every call — but nothing in the frontend calls it anymore.

What this means: Two heavy packages — `puppeteer-core` (~50MB) and `@sparticuz/chromium` (~100MB) — are installed and bundled for a route that no frontend button reaches. This is the single largest piece of dead weight in the codebase.

Recommendation: Remove `server/pdf.ts`, `server/report-pdf.ts` (client lib), `puppeteer-core`, and `@sparticuz/chromium` from `package.json`. Saves ~150MB from the production bundle and eliminates a Chromium process that was never being called.

Part 4 — Packages Installed But Not Used in Production

Package	Installed	Actually Used	Notes
`puppeteer-core`	Yes	No	Orphaned — see Part 3
`@sparticuz/chromium`	Yes	No	Orphaned — see Part 3
`html2pdf.js`	Yes	No	Not imported anywhere in the codebase. Zero references. Can be removed.
`axios`	Yes	No	Not imported anywhere in the codebase. Zero references. The template includes it by default. Can be removed.
`streamdown`	Yes	Partially	Used in `AIChatBox.tsx` and the template `Home.tsx` stub. `AIChatBox` is not mounted in any live route. `Home.tsx` is not rendered. Both are template scaffolding.
`@hookform/resolvers`	Yes	Uncertain	Part of the shadcn/ui form stack. May be used in form components. Low risk to keep.
`date-fns`	Yes	Uncertain	Part of the shadcn/ui calendar component. Low risk to keep.
`embla-carousel-react`	Yes	Uncertain	Used in `ui/carousel.tsx` shadcn component. Not used in any tab. Low risk to keep.
`react-day-picker`	Yes	Uncertain	Used in `ui/calendar.tsx` shadcn component. Not used in any tab. Low risk to keep.
`vaul`	Yes	Uncertain	Shadcn drawer component. Not used in any tab. Low risk to keep.
`input-otp`	Yes	Uncertain	Shadcn OTP input. Not used in any tab. Low risk to keep.
`cmdk`	Yes	Uncertain	Shadcn command palette. Not used in any tab. Low risk to keep.

The three clean cuts: `puppeteer-core`, `@sparticuz/chromium`, and `html2pdf.js`. These are confirmed unused and add significant bundle weight. `axios` is also confirmed unused and can be removed.

Part 5 — Template Scaffolding That Is Not Part of the Live App

These files exist in the codebase but are not rendered in any live route.

File	Status	Notes
`client/src/pages/Home.tsx`	Template stub — not rendered	The template default home page. `App.tsx` routes `/` directly to `DashboardLayout` + `HomeTab`. This file is never mounted. It imports `Streamdown` and `useAuth` for example purposes only.
`client/src/pages/ComponentShowcase.tsx`	Template stub — not rendered	A full component showcase page (shadcn/ui demos, AIChatBox demo, etc.). Not registered as a route in `App.tsx`. Never visible to any user.
`client/src/components/AIChatBox.tsx`	Template component — not mounted	Full-featured AI chat interface. Only referenced in `ComponentShowcase.tsx`, which is not a live route. The Christie AI Tab is a Sprint 5 horizon item — when that tab is built, this component will be used.
`client/src/components/DashboardLayoutSkeleton.tsx`	Template component — not mounted	Loading skeleton for auth checks. Not referenced anywhere in the live app.

These are not dead weight in the same way as unused packages. They are template scaffolding that will either be used (AIChatBox → Christie AI Tab) or can be cleaned up in a future sprint. They do not affect bundle size significantly and do not run any processes.

Part 6 — Legacy Integrations: What Was, What Is, What Is Not

This section addresses the question directly: Telegram, old Netlify site, old dashboard, old scheduled tasks.

Telegram: There is no Telegram bot code anywhere in this codebase. There is no Telegram token, no bot handler, no webhook. The Telegram bot (`ChristiesEHBot`) was part of the old Netlify build — it lived in `netlify/functions/whatsapp-inbound.js` and similar files. That entire architecture was replaced by the Manus replatform. **Telegram is gone. Nothing to cut.**

Old Netlify Site: The old site (`christiesrealestategroupeh.com` previously pointed to Netlify) has been replaced by the Manus-hosted platform. The Netlify functions that handled the morning brief, midnight brief, WhatsApp inbound, and send-brief are no longer running. The domain now points to the Manus deployment. There are no Netlify functions in this codebase — no `netlify.toml`, no `netlify/functions/` directory. **The old Netlify architecture is fully retired.**

Old Dashboard: The previous dashboard (the one with the Netlify JSON pipeline, the deal store, the tracked link logger) is gone. The PIPE tab in the current platform replaced it with a proper tRPC + MySQL database. **Nothing from the old dashboard is running.**

Manus Scheduled Tasks: I checked the sandbox for any Manus-level scheduled tasks (the `schedule` tool that fires prompts at intervals). There are none currently registered. The only scheduled processes are the two `node-cron` jobs inside the server (`0 8 * * *` and `0 20 * * *`), which are failing due to the Twilio credential issue.

Google Workspace / Google Drive connector: The `.user_env` file shows `GOOGLE_WORKSPACE_CLI_TOKEN=""` and `GOOGLE_DRIVE_TOKEN=""` — both empty. The Google Sheets in INTEL and PIPE are embedded as read-only iframes. No Google API credentials are in use. No write access to any sheet exists. This is correct and intentional.

GitHub connector: The `.user_env` file shows `GH_TOKEN=""` — empty. No GitHub remote is configured. The codebase lives in Manus version control only. The GitHub export (`christies-east-hampton` private repo) is a Sprint 5 item.

Part 7 — The Two Domains

Domain	Points To	Status
`christies-dash-acqj9wc4.manus.space`	Manus dev preview	Live — development URL
`www.christiesrealestategroupeh.com`	Manus production	Live — production domain

Both domains are managed inside the Manus platform. There is no separate Netlify deployment, no Vercel, no Railway. One platform, one codebase, two URLs (dev + prod).

Part 8 — What to Cut, What to Keep, What to Watch

Cut (confirmed dead weight, no risk)

Item	Why	How
`puppeteer-core`	Orphaned route — no frontend calls it	Remove from `package.json`, delete `server/pdf.ts`
`@sparticuz/chromium`	Same as above	Remove from `package.json`
`html2pdf.js`	Zero references in codebase	Remove from `package.json`
`axios`	Zero references in codebase	Remove from `package.json`
`server/pdf.ts`	Orphaned route file	Delete file
`client/src/lib/report-pdf.ts`	Orphaned client lib — nothing calls it	Delete file

Estimated bundle savings: ~150–180MB from removing Puppeteer + Chromium alone.

Keep (actively used or needed soon)

Item	Why
`twilio`	WhatsApp scheduler — needs credential fix, not removal
`node-cron`	Scheduler backbone — keep, fix credentials
`jspdf` + `jspdf-autotable`	Powers all 6 PDF export buttons in IDEAS
`d3`	Powers the Paumanok plate map in MAPS tab
`recharts`	Powers the donut chart in MARKET tab
`framer-motion`	Used in tab transitions and UI animations
`AIChatBox.tsx`	Will be used when Christie AI Tab is built in Sprint 5
`streamdown`	Will be needed when AIChatBox is mounted

Watch (template scaffolding — not urgent)

Item	Decision
`client/src/pages/Home.tsx`	Can be deleted — never rendered. Or repurposed as a public landing page if the platform ever goes client-facing.
`client/src/pages/ComponentShowcase.tsx`	Can be deleted — never rendered. Template reference only.
`client/src/components/DashboardLayoutSkeleton.tsx`	Keep — useful if auth loading states are ever added. Zero overhead.

Summary

The platform is lean. The only genuine dead weight is the Puppeteer/Chromium stack (~150MB) that was orphaned when the PDF download button was switched to jsPDF. Everything else is either actively used, needed for Sprint 5, or low-overhead template scaffolding.

The most urgent operational issue is not a cleanup item — it is the Twilio credential failure. The WhatsApp scheduler is wired correctly and will work the moment valid credentials are

confirmed in the environment.

There is no Telegram. There are no old Netlify functions. There are no background Manus tasks running outside the server. The platform is one codebase, one server, one database, two domains.